

למידת מכונה

—

סיווג אודיו לספרות

יאיר הורביץ 207377730

עדי פסח 326627635



Free Spoken Digit Dataset (FSDD)

FSDD הוא מאגר נתונים חינוכי של קבצי שמע, המיועד למחקר וללמידת מכונה בתחום זיהוי דיבור. המאגר כולל הקלטות של ספרות באנגלית (0–9), בדומה ל-MNIST המפורסם בתחום התמונות – אך כאן מדובר בגרסת שמע.

מאפיינים עיקריים

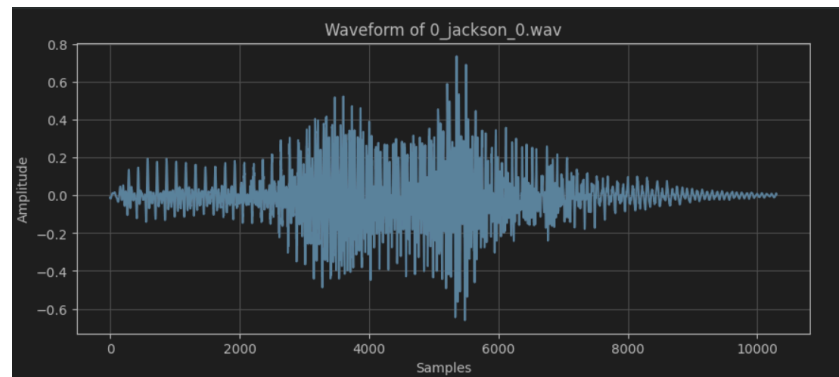
- **מספר דוברים:** 6 דוברים שונים.
- **כמות הקלטות:** 3,000 קבצים בסך הכול (50 הקלטות לכל ספרה מכל דובר).
- **פורמט:** כל קובץ הוא בקובץ WAV סטנדרטי, שבו הנתונים מיוצגים כסיגנל אודיו (גל קולי).
- המאגר מתאים במיוחד לניסויים בלמידת מכונה, עיבוד קול ודיבור (Speech Processing).
- **ייצוג האות:** ההקלטות הן **גלי קול ספרתיים (waveform)**, שנדגמו בקצב של **8kHz** (כלומר 8,000 דגימות בשנייה).
- **שפה:** אנגלית.
- **מבנה שמות הקבצים:** {ספרה}_{שם_דובר}_{אינדקס}.wav, לדוגמה:

7_jackson_32.wav.

איך נראה דאטא?

תחילה הוקלט אות קול (Waveform) באמצעות מיקרופון. כל גרף מציג את האות הקולי (הספרה שנאמרה בקול) כגל קול לאורך ציר הזמן:

- **ציר X (דגימות):** מייצג את מספר הדגימות בזמן. מאחר שההקלטות נדגמו בקצב של 8kHz, כל 8,000 נקודות מייצגות שנייה אחת של שמע.
- **ציר Y (עוצמה):** מייצג את עוצמת האות (הסטייה מהשקט). ערכים חיוביים ושליילים מצביעים על תנודות הגל הקולי.

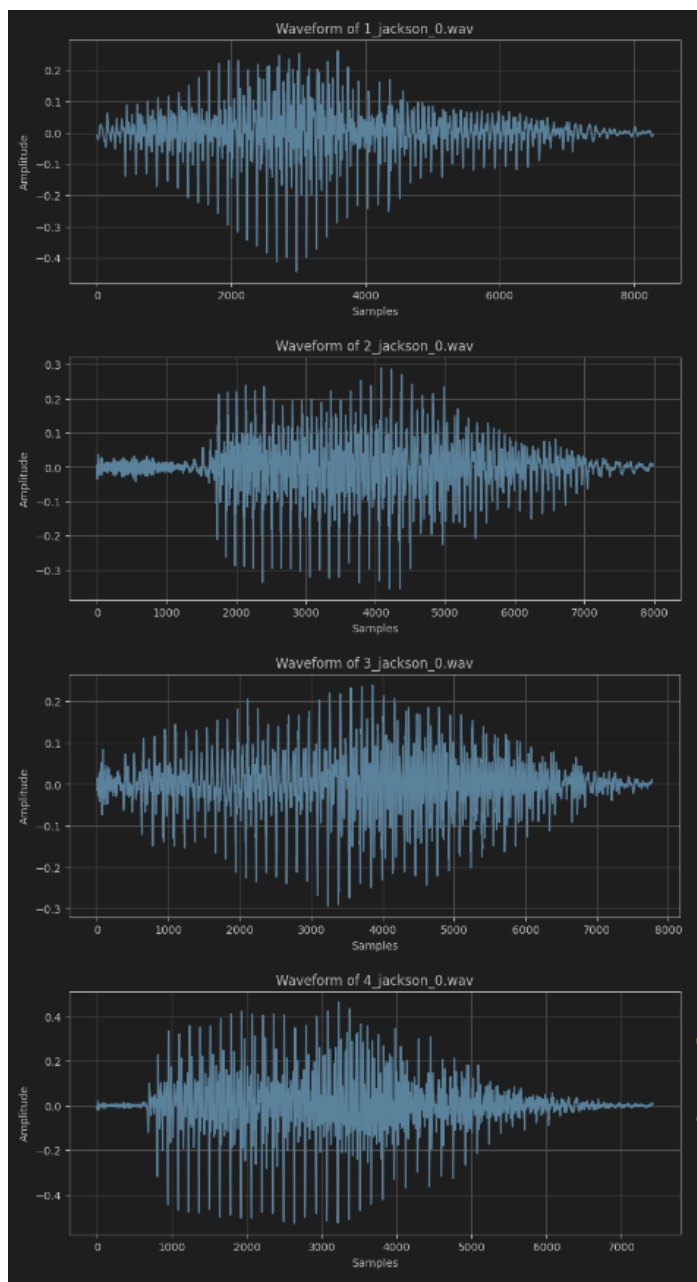


בגרפים ניתן יהיה לראות כי:

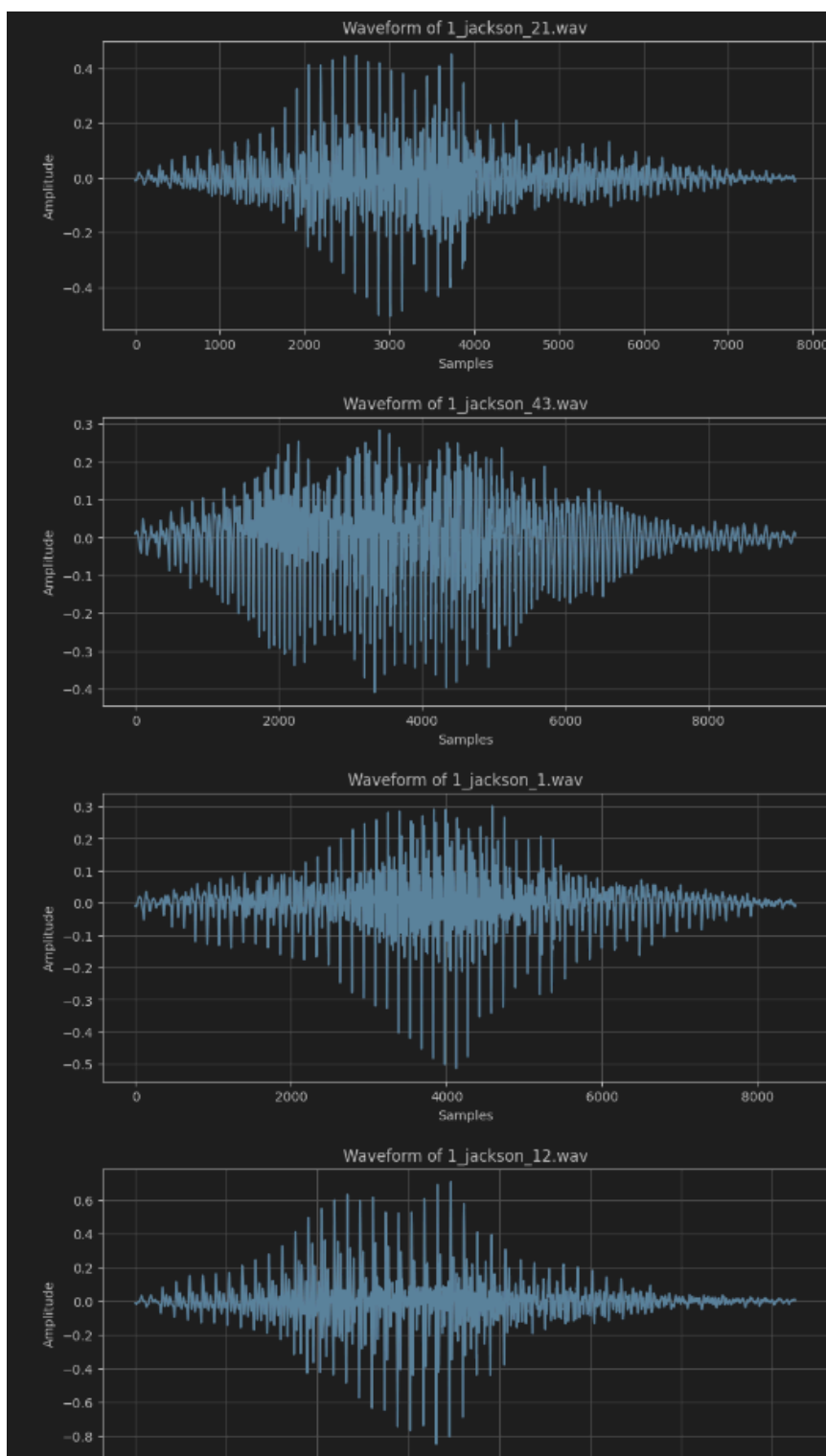
- לכל ספרה יש צורה ייחודית של גל קול, אפילו קיימת שונות בין חזרות של אותו דובר.
- בחלק מההקלטות, יש שקט בתחילה ו/או סוף ההקלטה, ובחלק אין (עקב חיתוך השקטים שבוצע מראש במאגר שלא מושלם ולא אחיד).
- העוצמות והמבנה הפנימי של הגל משקפים את התכונות של הספרה שנאמרה.

הצגת גרפים של הקלטות

על פי הגרפים למטה ניתן לראות את ההקלטות שונות של **Jackson** שאומר את הספרות 1 עד 4 (כולל). ניתן לראות כי יש הבדלים בצורות של הגרפים של כל אחת מהספרות, על אף שזה נאמר על ידי אותו בן אדם. בנוסף, יש הבדלים בשקט בהתחלה ובסוף. לדוגמה, בהתחלה של ספרה 2 יש שקט ארוך בשאר הספרות כמעט ואין שקט בהתחלה.



על פי הגרפים למטה ניתן לראות הקלטות שונות של **jackson** אומר את **הספרה 1**. כמו כן ניתן לראות שלמרות שאלו הקלטות של **אותו אדם על אותה הספרה**, עדיין יש הבדלים רבים בין הגרפים וקשה להבחין מיד שזה אותה הספרה. אך עדיין, ניתן לראות את המבנה הפנימי שיש בו דמיון בין ההקלטות.



המרת קטעי הקול לפיצרים:

כעת נרצה לעבור מהייצוג הגרפי עם אלפי דגימות לפיצ'רים שנוכל לעבוד איתם וללמוד מהם.

בטרם חילוץ הפיצ'רים, יש צורך לחלק את הסיגנל הקולי לחלונות קצרים וקבועים. הסיבה לכך היא שקול אנושי הוא **אות לא יציב** (non-stationary) – התדרים והעוצמה משתנים לאורך הזמן כפי שניתן לראות מהגרפים. במילים אחרות, אות הוא יציב אם ניתן להגדיר אותו עם פולינום עם מעט מקדמים. עם זאת, אם נתמקד בפרק זמן קצר מאוד, ניתן להניח שהאות כמעט יציב (stationary), ולכן אפשר לנתח אותו בצורה מתמטית מדויקת יותר.

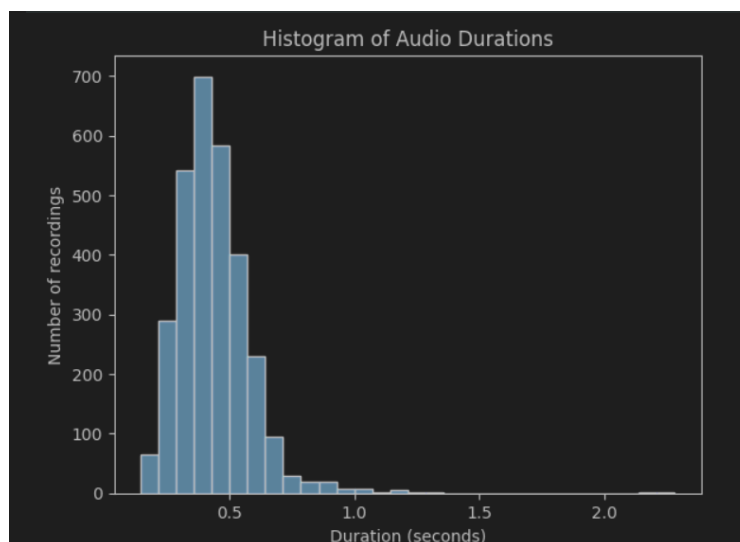
בפרויקט זה חילקנו את הסיגנל לחלונות באורך **40 מילי-שניות (320 דגימות בקצב דגימה של 8kHz)**, עם צעד של **10 מילי-שניות (80 דגימות)** בין תחילת חלון אחד לתחילת הבא אחריו. משמעות הדבר היא שקיימת חפיפה של כ-75% בין חלונות סמוכים.

הסבר לבחירת הגדלים

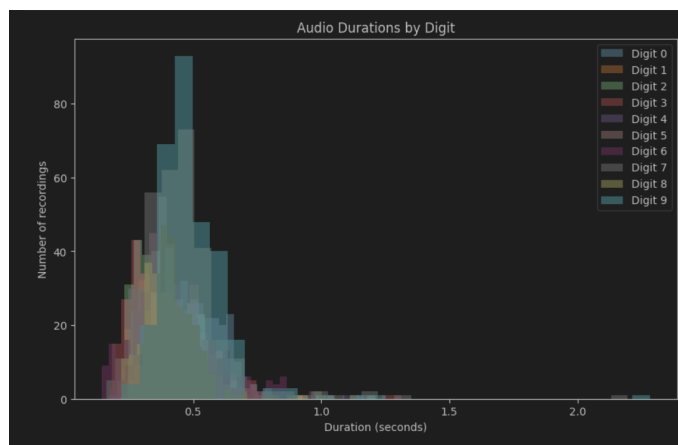
- **אורך חלון (length) 40ms:**
חלון קצר מדי (למשל 10ms) לא מספיק כדי ללכוד מידע על התדרים הנמוכים שבקול האנושי. לעומת זאת, חלון ארוך מדי (80–100ms) יגרום לכך שהאות כבר לא ייחשב יציב. הבחירה ב-40ms היא איזון טוב, והיא גם ערך מקובל בעיבוד דיבור.
- **צעד בין חלונות (stride) 10ms:**
צעד קטן מבטיח כיסוי צפוף של הסיגנל ושמירה על רציפות המידע, גם אם יש שינויים מהירים בהגייה. הערך של 10ms הוא סטנדרט נפוץ בתחומי זיהוי דיבור, והוא מאפשר שהמידע לא "יאבד" בין חלון לחלון.

גודל זהה:

ההקלטות במאגר שונות באורכן, ולכן כדי לאפשר שימוש במודלים של למידת מכונה יש צורך לייצג אותן באורך קבוע. לשם כך השתמשנו בריפוד (Padding) או חיתוך (Trim) כדי שכל המטריצות של הפיצ'רים יהיו באותו גודל. כעת ניסינו לדעת איזה גודל יהיה אידיאלי שנשנה אליו. יצרנו גרף המתאר לנו את כמות הזמן של כל הקלטה:



בנוסף רצינו לראות כיצד מתחלק האורכים על פי ספרות כדי שלא יצא מצב שכל ההקלטות הקצרות יהיו של ספרה אחת או הארוכות ואז ייתכן ויהיה מספרים שממש יפגעו משינוי זמן ההקלטות. ובאמת נראה מהתמונה שיש פיזור טוב ואחיד של כל הספרות.



```

Min: 0.14s
Max: 2.28s
Mean: 0.44s
Median: 0.42s
Standard Deviation: 0.15s

```

נשים לב שפעולת padding תוסיף שקט בסוף ההקלטה ולכן אם נאמרה ספרה כלשהי, גם לאחר padding עדיין נאמרה הספרה ולכן פעולה זו לא הורסת, לעומת זאת trimming, עלול לקטוע הקלטה באמצע שנאמרת ספרה ולכן זוהי פעולה שכן עלולה לפגוע. על כן, **נעדיף לקבוע את נקודת החיתוך שלנו יחסית מאוחר יותר**, כי זה יגרום לכך שיהיה פחות trimming והסתברות שהקלטה תקטע באמצע אמירת הספרה תהיה נמוכה יותר. אמנם נקודה מאוחרת אומרת שנוסיף יותר padding, אך כפי שהסברנו זוהי פעולה שלא הורסת את הlabel.

כפי שניתן לראות הממוצע הוא 0.44 שניות והחציון הוא 0.42 שניות להקלטה וכן ניתן לראות שלאחר 0.9 נותרו ממש מעט הקלטות כך שהחלטנו לחתוך ב-0.9 וכך יהיה לרוב מוחלט של ההקלטות כל הייצוג וכן הריפוד שאנו מוסיפים יהיה גדול אבל לא ענק וכך ניסינו ליצור איזון מסוים בין הריפוד לחיתוך אם הפסד מינימלי לדאטה.

כפי שהסברנו מקודם על חלוקת הגרף להרבה חלונות אך עדיין אנו רוצים לחלץ מתוך החלונות הללו פיצרים רלוונטים שיעזרו לנו לחזות נחלץ 13 פיצרים של מקדמי MFCC - הפונקציה בחלון (מספר סטנדרטי ומקובל במחקר ועיבוד דיבור).

הניסיון הראה שהכמות 13 היא איזון טוב: מצד אחד מספיק כדי לייצג את מאפייני התהודה (formants) של הקול, מצד שני לא כבד מדי חישובית ולא מוסיף רעש מיותר. אם ניקח יותר מדי מקדמים (למשל 40), נקבל גם פרטים עדינים בתדרים גבוהים שהאוזן פחות רגישה אליהם – והם נוטים להיות פחות יציבים ורועשים. אם ניקח פחות מדי (למשל 3–5), לא נצליח לייצג את המידע הדרוש להבחנה בין ספרות שונות.

ארגון הפיצ'רים לצורך למידה:

השלב הבא הוא באמת איך לארגן את הפיצ'רים לצורך למידה. יש כמה גישות מקובלות אנו השתדלנו להשתמש בכלם ולראות מה עדיף:

1. ממוצע וסטיית תקן (Aggregate)

- לוקחים את כל הווקטורים שמתקבלים מכל החלונות במשך ההקלטה ועושים ממוצע וסטיית תקן.
- יוצא וקטור אחד באורך 13 שמייצג את הממוצעים של כל ההקלטות ווקטור נוסף באורך 13 שמייצג את סטיית התקן של כל ההקלטות - סה"כ וקטור של 26 תאים.
- יתרון: פשוט, מתאים למודלים קלאסיים, שומר על מימדים נמוכים, גודל קבוע.
- חיסרון: מאבד מידע סדרתי, לא יודעים איך האות השתנה לאורך הזמן.

2. וקטור שטוח (Flatten)

- לוקחים את כל הווקטורים ברצף ומחברים אותם לווקטור אחד ארוך.
- לדוגמה: אם יש 50 חלונות וכל חלון באורך 13 - יוצא וקטור באורך 650 (בפועל על פי כמות החלונות בהתאם לנקודת חיתוך ב-0.9 יחד עם גודל החלון והצעד יוצא 1183 מימדים להקלטה).
- יתרון: שומר את כל המידע.
- חיסרון: בעייתי אם אורך ההקלטות שונה (צריך padding/trimming), וגם הממד נהיה מאוד גדול.

3. סדרה (Sequence)

- מתייחסים לרצף הווקטורים כאל סדרה לאורך הזמן.
- מתאים למודלים שיוזעים לעבוד עם נתונים סדרתיים כמו במקרה שלנו.
- יתרון: שומר את הדינמיקה הסדרתית של האות.
- חיסרון: דורש מודל מורכב יותר וזמן חישוב גבוה יותר.

4. מטריצה (CNN Input)

- מארגנים את הפיצ'רים כמטריצה: ציר אחד = זמן (מספר החלונות), ציר שני = מספר הפיצ'רים (13).
- אפשר להכניס ישירות ל-CNN, כי זה מטריצה (גובה = 13, רוחב = מספר החלונות) ויש חשיבות לקרבה בין ערכים - זה מסמל שינויים מקומיים באות.
- יתרון: מאפשר לנצל טכניקות של ראיית מחשב (Convolution).
- חיסרון: בעייתי אם אורך ההקלטות שונה (צריך padding/trimming), וגם הממד נהיה מאוד גדול, רק שהפעם הוא במטריצה.

הדגמות ויזואליות:

כעת נרצה להמחיש כמה שונה כל ספרה על ידי גרף. המטרה של סיווג היא להבדיל בין מחלקות (כאן – הספרות 0–9). אם בגרף רואים שהפיצ'רים של ספרה מסוימת (למשל "0") מתרכזים באזור אחר במרחב מאשר ספרה אחרת (למשל "9"), זה מראה שהאלגוריתם יוכל ללמוד להבדיל ביניהן.

הדגמה ויזואלית לקורא:

גרף (למשל PCA דו־ממדי ותלת-מימדי) מאפשר להבין באופן אינטואיטיבי את ההבדלים בין הספרות. זה הרבה יותר ברור מאשר להסתכל רק על מטריצות מספרים.

השוואת ייצוגים שונים:

נוכל להשוות בין שיטות ייצוג שונות לדוגמא על **Aggregate** לעומת **Flatten**, נוכל לראות איזה ייצוג מפריד טוב יותר בין הספרות. וזה נתן הצדקה לבחירה שלנו בשיטת הייצוג.

חשיפה לקשיים:

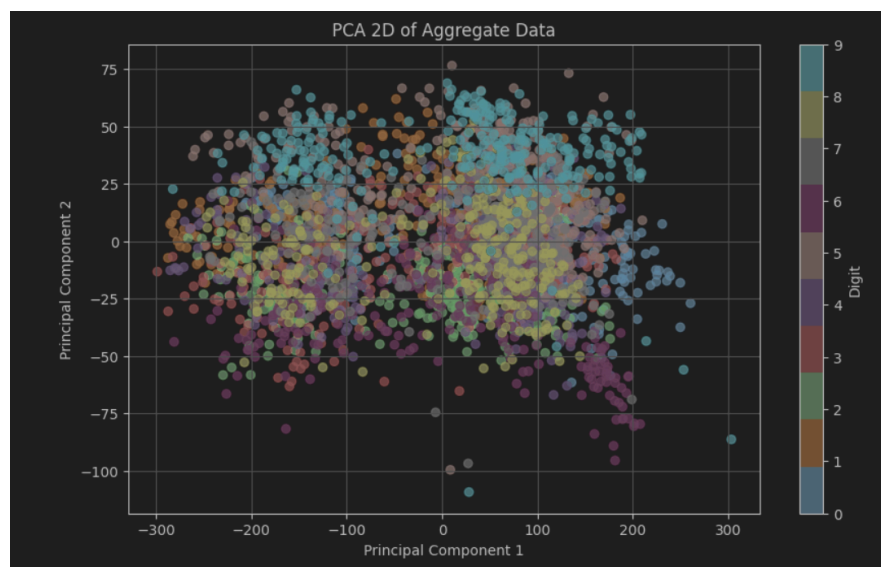
אם יש ספרות שקשה להבדיל ביניהן (למשל "3" ו-"8"), זה יופיע בגרף כקבוצות שמתערבבות. כך נבין מראש איפה המודל עלול לטעות.

קצת קשה להמחיש 26 ממדים לכן בעזרת PCA (צמצום מימדים) נוכל להראות זאת בגרף:

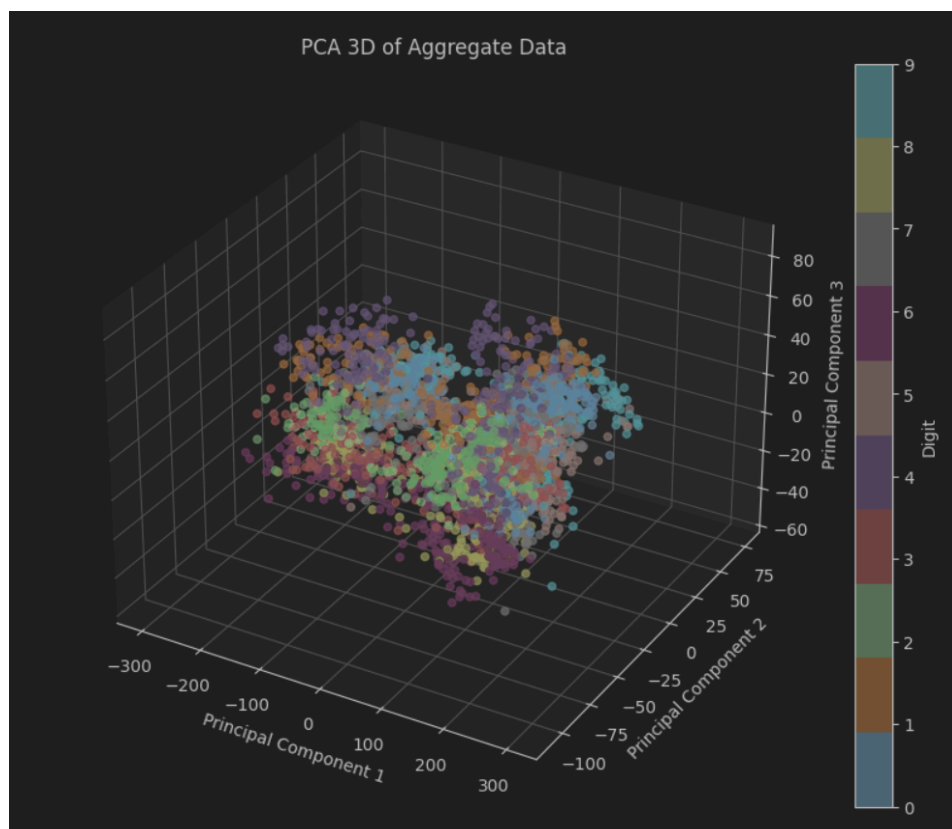
-Aggregate

נראה דבר ראשון את הגרף בשיטת ה**Aggregate** כאשר כל צבע מסמל ספרה והמיקום נקבע בעזרת צמצום המימדים על הפיצרים ניתן לראות כי על אף שצמצמנו את 26 פיצרים ל 3 ו 2 עדיין יחסית המספרים מחולקים בסביבות מקומיות אך עדיין מעורבבים זה בזה, מה שמקשה על ההפרדה.

דו ממדי:



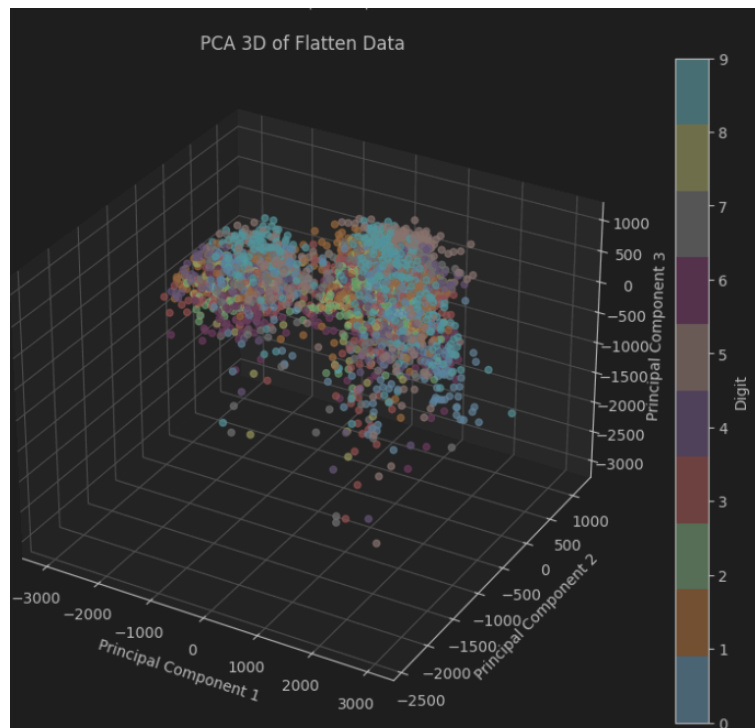
תלת ממדי:



-Flatten

כעת נמחיש בעזרת **Flatten** היינו מצפים שהתוצאות יהיו טובות יותר שהרי אנו מאבדים פחות מידע אך באופן מפתיע נראה שהייצוג הזה אפילו מפריד מעט פחות טוב, למרות שהוא יותר עשיר ומפורט ואף כולל בתוכו מידע בשביל להסיק את הנתונים של Aggregate.

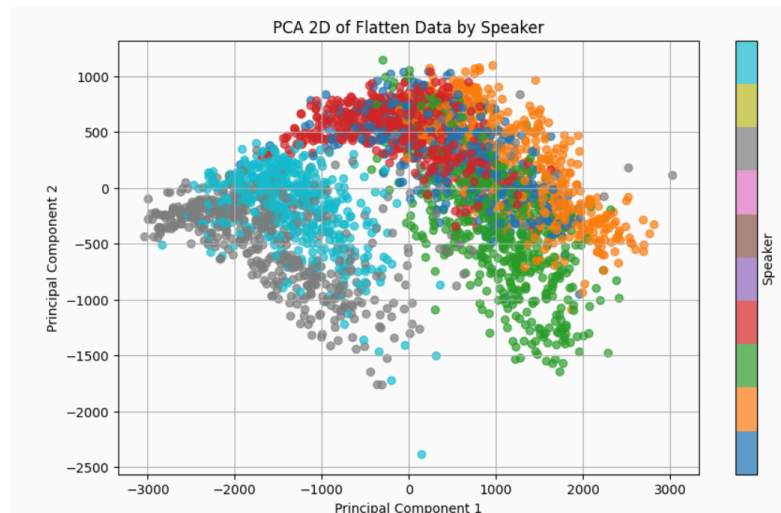
ההשערה לסיבה לכך היא שלעומת פעם קודמת, שהורדנו 24 מימדים, הפעם אנחנו מורידים 1181 מימדים. סדר הגודל שונה לגמרי ולכן הייצוג באמצעות PCA של Flatten יהיה משמעותית פחות "מייצג" את הגרף האמיתי לעומת של Aggregate.

דו ממדי:**תלת ממדי:**

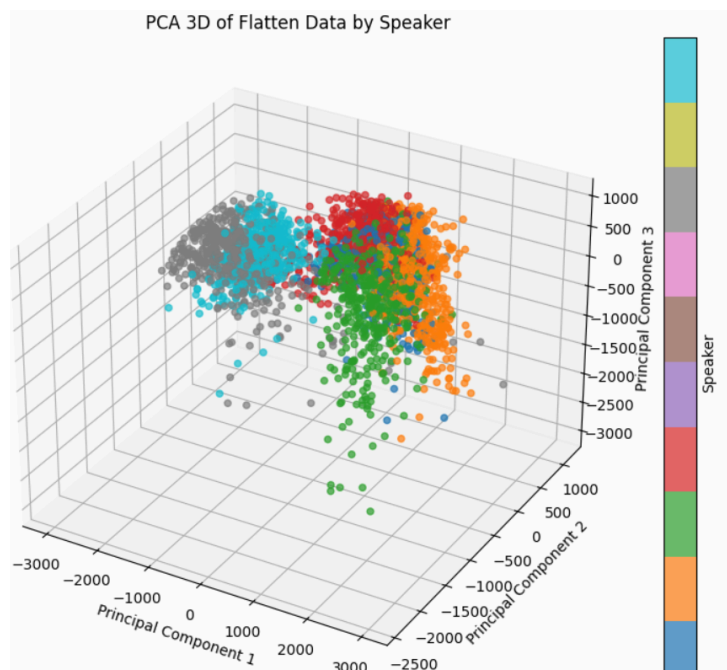
speakers

בנוסף, רצינו לראות את ההבדלים בין הדוברים וגם את ההבדלים בין כל דובר לעצמו (בהקלטות שונות). על כן, כמו קודם, ביצענו PCA על Flatten רק הפעם, הנקודות צבועות לפי הדובר של אותה ההקלטה. מהגרפים האלה, ניתן לראות שלעומת הספרות שמאוד מפוזרות גם זו וגם בעצמן, הדוברים מאוד ממוקדים, כלומר הקלטות שונות של אותו הדובר הן באותה הסביבה. בנוסף אנו רואים שהדוברים מתחלקים לכמה איזורים, ולכן כנראה שאם במהלך האימון אנו מכירים רק דוברים מסוימים, ייתכן שיהיה קשה לחזות ערכים של דוברים באיזורים שלא הכרנו, לדוגמה אם נתאמן על הדוברים התכלת והאפור, יהיה קשה לחזות לגבי השאר.

דו ממדי:



תלת ממדי:



המטרה שלנו היא לבנות מודלים שיסווגו את קטעי הקול לספרות. נרצה להיות מסוגלים שהמודלים יוכלו לסווג קטעי קול גם של דוברים שהם התאמנו עליהם וגם להיות מסוגלים לסווג קטעי קול של דוברים חדשים. נבחן את ההבדל ברמת הקושי בין המשימות האלו.

בעולם האמיתי, נרצה להיות מסוגלים לסווג דוברים שלא התאמנו עליהם, שכן אחרת נצטרך דאטא רב מכל דובר חדש ולאמן מודל חדש. לעומת זאת, אם נבנה מודל שמסוגל להיות כללי ולסווג ספרות מדוברים חדשים, נוכל להשתמש בו ללא צורך בדאטא נוסף או אימון נוסף על כל אדם בעולם. מאחר ובדאטאסט שלנו יש רק 6 דוברים, סביר שיהיה קשה למודל להכליל את עצמו לקולות של כל העולם, אבל נרצה לבחון כמה הוא יכול תחת המגבלות. בנוסף, נרצה לראות איזה גישת ייצוג של הפיצ'רים טובה יותר ללמידת מכונה ממה שראינו ואת ההבדלים ביניהם.

מודלים

המודלים שבחרנו הם:

1. KNN (k-Nearest Neighbors)

מודל פשוט ולא פרמטרי, מתאים במיוחד לבעיות שבהן יש מרחקים טבעיים במרחב הפיצ'רים. נותן baseline חזק לבדיקה ראשונית: אם הדאטא מיוצג טוב (כמו Aggregate עם KNN, mean+std), יחסית יפה. קל לפרש את ההשלכות של התוצאות שלו: טעויות נובעות מערבוב בין דוגמאות "קרובות מדי". → ממנו לומדים כמה ייצוג הפיצ'רים לבדו מספיק ללא מודל מורכב.

2. SVM (Support Vector Machine)

חזק במרחבים בעלי מימדיות גבוהה (כמו Flattened). מתאים במיוחד ל-datasets לא מאוד גדולים, כמו FSDD (הדאטאסט שלנו). → ממנו לומדים האם יש גבולות החלטה ברורים בין הספרות, והאם המידע מספיק להפרדה.

3. Random Forest

מודל אנסמבל מבוסס עצי החלטה (ע"פ הצבעות של העצים), חזק בבעיות שבהן יש תכונות לא ליניאריות ושילובי פיצ'רים. מתמודד טוב עם דאטא רועש ומצבים שבהם חלק מהפיצ'רים לא אינפורמטיביים (לא מועילים). נותן גם מדד של חשיבות פיצ'רים, שמעניין במיוחד כדי לראות אילו מקדמי MFCC מועילים יותר. → ממנו לומדים על מורכבות הקשרים בין הפיצ'רים, ואילו תכונות משמעותיות.

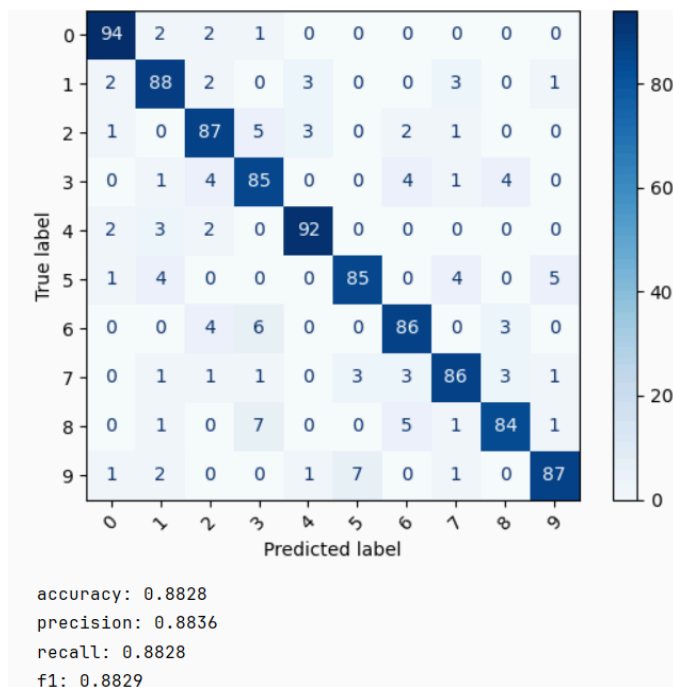
נשים לב שמאחר והדאטא שלנו מאוזן, כלומר לכל ספרה יש אותה כמות הקלטות אז accuracy יכול לתפקד כמטריקה טובה לאבלואציה של המודל.

סיווג ספרות- דוברים מוכרים

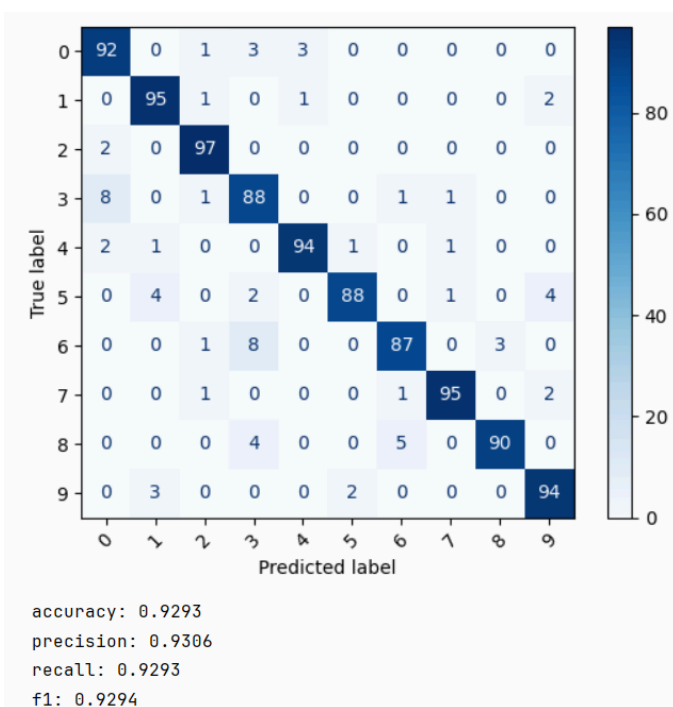
הדבר הראשון שרצינו לנסות הוא לבדוק כמה קשה לסווג קולות של דוברים שכבר התאמנו עליהם.

ראשית, נבדוק זאת בתנאים הפשוטים ביותר, לפי הדאטא ב-AGGREGATE. לכן חילקנו את הדאטא ל- $train/test$ באופן רנדומלי לפי 67/33 ובדקנו את המודלים הקלאסיים:

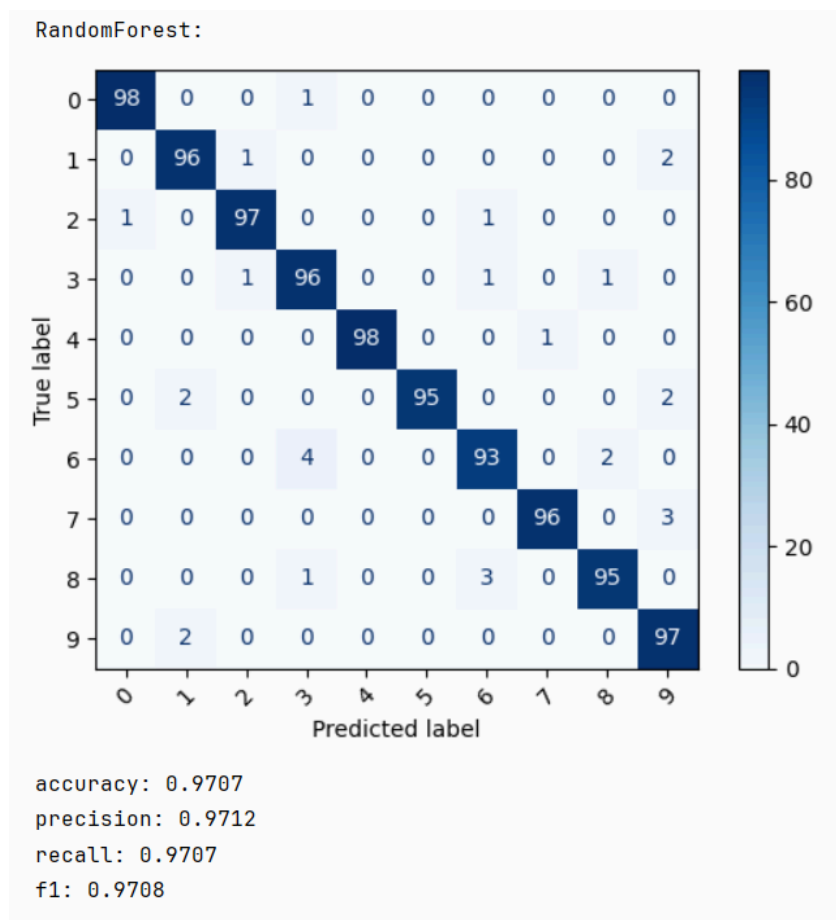
:KNN



:SVM



:RandomForest



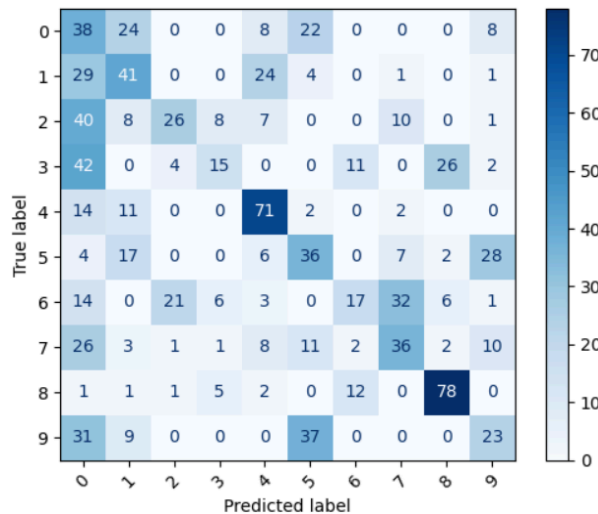
כפי שניתן לראות, כל המודלים הצליחו להשיג דיוק גבוה מאוד בקלות, במיוחד random forest שהצליח 97% ואפילו KNN פשוט הצליח 88%, מה שמעיד על רמת קושי נמוכה בשביל סיווג דוברים מוכרים. נרצה להשוות זאת לעומת סיווג דוברים שהמודלים לא הכירו באימון.

סיווג ספרות- דוברים שאינם מוכרים**:AGGREGATE**

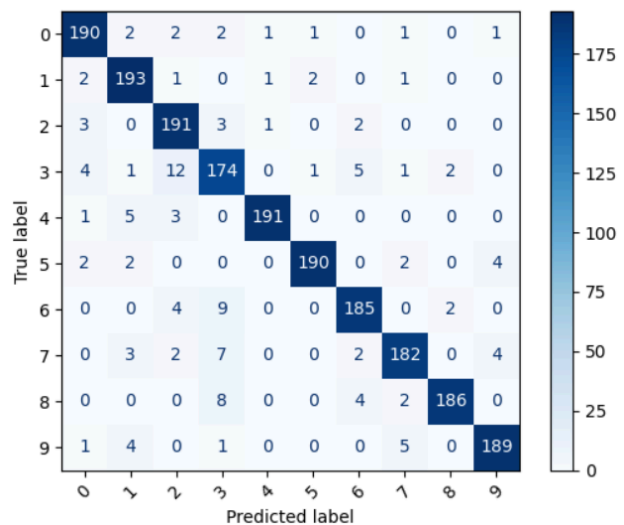
עבור סיווג דוברים שהמודלים לא הכירו באימון, השתמשנו באותו הדאטא (על AGGREGATE), אך הפעם במקום לחלק רנדומלית, הפרדנו 4 דוברים לאימון ו20 לטסט (בכך ההתפלגות train/test נשמרת) והשאר בדיוק כמו קודם. בכך, למודלים יהיה אותו המידע כמו פעם קודמת, אך הפעם הם ייבחנו על דוברים שהם לא הכירו. אלו תוצאות המודלים גם על הדאטא שהתאמנו עליו (train) וגם על הדאטא שלא ראו (test):

:KNN

Test results:



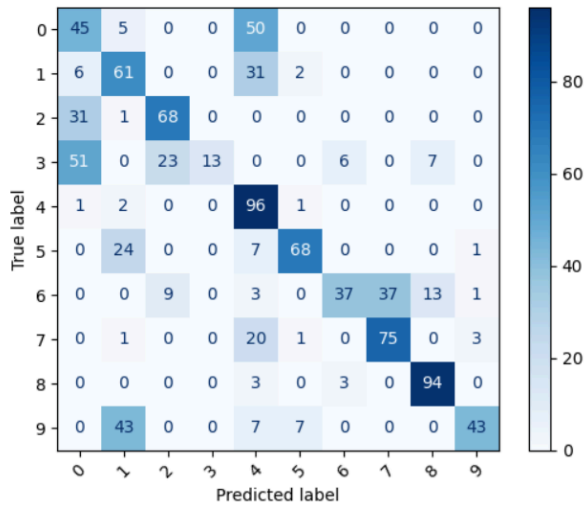
Train results:



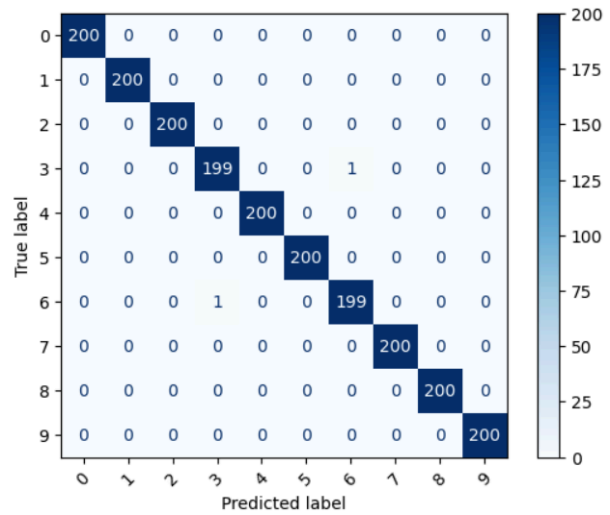
Metric	Train	Test
accuracy	0.9355	0.3810
precision	0.9366	0.4118
recall	0.9355	0.3810
f1	0.9357	0.3745

:SVM

Test results:



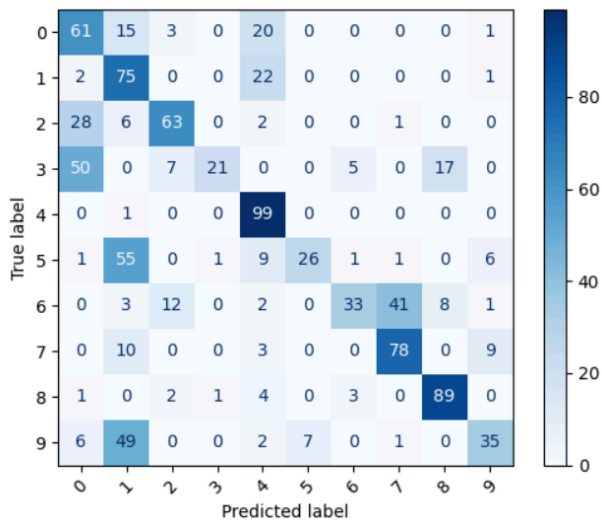
Train results:



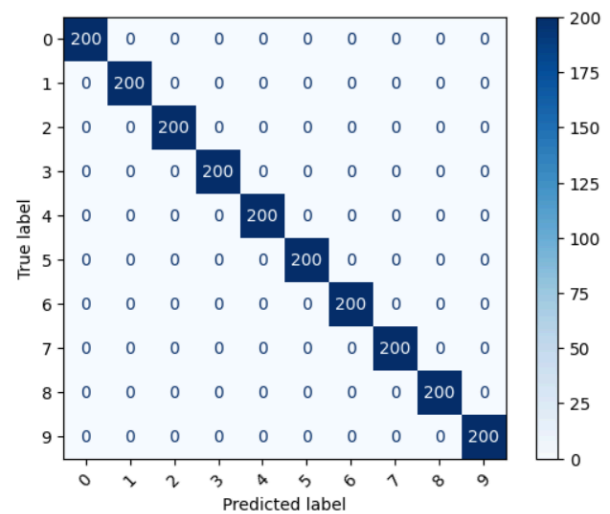
Metric	Train	Test
accuracy	0.9990	0.6000
precision	0.9990	0.6959
recall	0.9990	0.6000
f1	0.9990	0.5849

:RandomForest

Test results:



Train results:



Metric	Train	Test
accuracy	1.0000	0.5800
precision	1.0000	0.6658
recall	1.0000	0.5800
f1	1.0000	0.5584

מודל	דוברים מוכרים	דוברים חדשים	הפרש
KNN	0.88	0.38	0.5
SVM	0.92	0.6	0.32
Random Forest	0.97	0.58	0.39

הפעם הביצועים היו נמוכים בפער בטסט. KNN השיג 38% (לעומת 88% מקודם), ושאר המודלים התקרבו ל-60%. באותו הזמן, כולם השיגו באיזור 100% באימון, מה שמעיד על overfitting רציני. כלומר, המודלים התאימו את עצמם יותר מדי על הדוברים המוכרים ולא הצליחו להכליל לדוברים חדשים. על כן ניתן להסיק שהמשימה של לסווג דוברים חדשים קשה בהרבה, במיוחד בהינתן הדאטא המוגבל של 6 דוברים בלבד. לכן מעתה נתרכז בסיווג דוברים חדשים, שכן סיווג דוברים מוכרים זה גם פחות שימושי וגם נפתר בקלות מאוד.

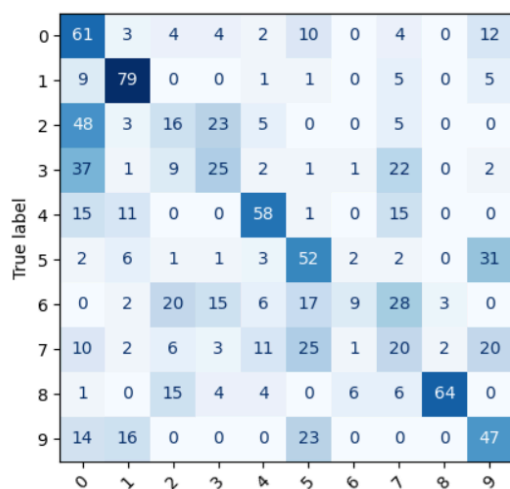
-FLATTEN

כעת נבדוק כיצד המודלים יבצעו כאשר יקבלו את FLATTEN_DATA במקום ה AGGREGATE_DATA. כאן, במקום לקבל מידע כללי על כל ההקלטה, ניתן מידע על כל ההקלטה. אז המידע כעת הרבה יותר עשיר ומפורט, אך גם עם מימדים גבוהים במיוחד (1183 מימדים), ובהינתן 2000 דוגמאות לאימון, עלול להיווצר overfitting. עקב ה curse of dimensionality.

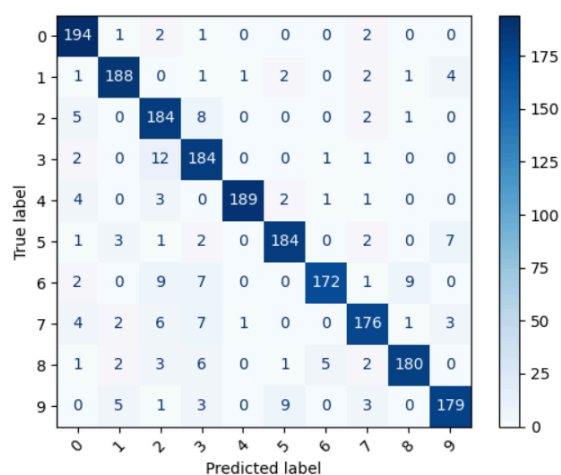
ראשית נבחן את אותם המודלים, על דוברים לא מוכרים, אך הפעם נתן להם את הדאטא בFLATTEN.

:KNN

Test results:



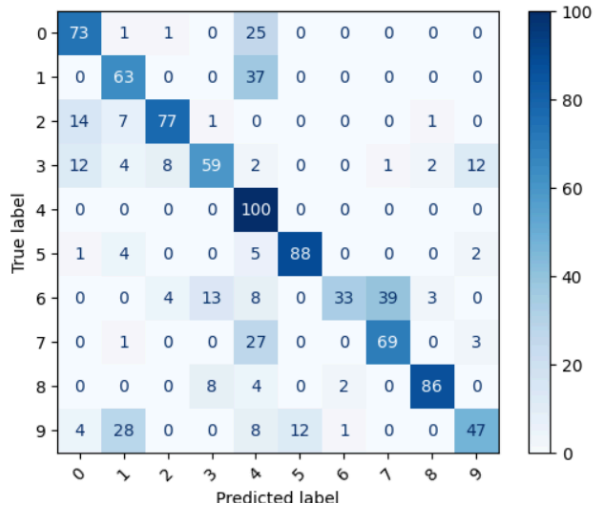
Train results:



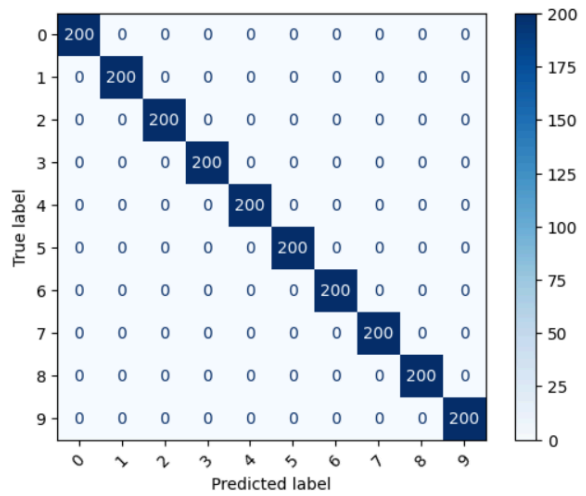
Metric	Train	Test
accuracy	0.9150	0.4310
precision	0.9176	0.4531
recall	0.9150	0.4310
f1	0.9154	0.4184

:SVM

Test results:



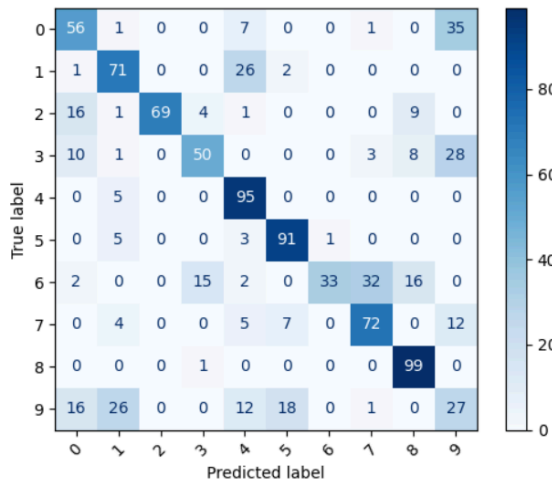
Train results:



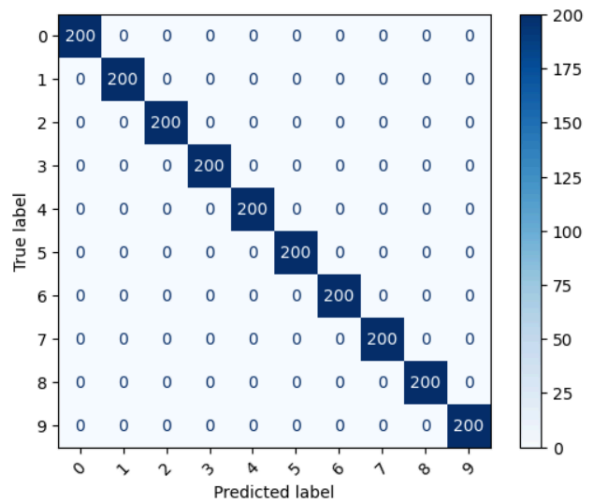
Metric	Train	Test
accuracy	1.0000	0.6950
precision	1.0000	0.7431
recall	1.0000	0.6950
f1	1.0000	0.6911

:RandomForest

Test results:



Train results:



Metric	Train	Test
accuracy	1.0000	0.6630
precision	1.0000	0.6938
recall	1.0000	0.6630
f1	1.0000	0.6520

מודל	Aggregate Accuracy	Flatten Accuracy	שיפור
KNN	0.38	0.43	0.05
SVM	0.60	0.695	0.095
Random Forest	0.58	0.663	0.083

ניתן לראות שאכן הביצועים של כל המודלים ישתפרו, מה שמעיד על כך שלהשתמש בנתונים מכל ההקלטה תורם הרבה יותר מאשר לקחת ממוצע של ההקלטה, אפילו עם KNN.

נשים לב שזה מתנגד לסרטוטי PCA מהניתוח הראשוני, מה שמעיד על כך שכנראה גרפי PCA היו מטעים מעט עקב הורדת המימדים הקיצונית יותר עבור flatten, מ-1183 מימדים ל-2-3 מימדים, לעומת ה-aggregate שהוריד רק 26 מימדים ל-2-3 מימדים.

בנוסף, ה-KNN השתפר מעט יחסית לשאר המודלים, ו-SVM השתפר בצורה המיטבית ביותר. מה שמעיד על איכות ההתמודדות של המודלים עם מימדים גבוהים. כמצופה, KNN השתפר בצורה מינימלית יחסית, בעוד ש-SVM ו-Random Forest הפיקו תועלת רבה יותר מהמעבר לנתונים ב-Flatten.

הסיבה לכך היא שמודלים שונים מתמודדים באופן שונה עם מימדים גבוהים:

- **KNN** סובל מ-*curse of dimensionality*: ככל שעולים במימדים, כל הנקודות נעשות "רחוקות" זו מזו, ולכן מדד המרחק (כמו Euclidean) נהיה פחות אינפורמטיבי (פחות מועיל). המשמעות היא שהשכנים הקרובים כבר לא בהכרח באמת קרובים, ולכן היכולת להבחין בין ספרות מתערערת. זה מסביר את השיפור הקטן בלבד ב-KNN.
- **SVM** לעומת זאת מותאם מאוד לעבודה במרחבים בעלי מימדיות גבוהה. האלגוריתם מחפש *hyperplane* (היפר-מרחב) שמפריד את הדאטה, וכאשר יש יותר פיצ'רים, מספיק שכמה מהם מפרידים היטב ו-SVM יודע לתת פחות חשיבות לאלה שלא. לכן השיפור של SVM היה המשמעותי ביותר.
- **Random Forest** מתמודד גם הוא יחסית טוב עם מימדים גבוהים, מאחר שכל עץ ביער בוחר תת-קבוצה שונה של פיצ'רים. כך גם אם חלק מהפיצ'רים לא מועילים או אפילו מבלבלים, עדיין קיימת אפשרות שעצים אחרים ינצלו פיצ'רים אינפורמטיביים ויתרמו לניבוי. עם זאת, ריבוי פיצ'רים עלול להוביל לפיצול יתר (overfitting), ולכן השיפור שלו בינוני – יותר טוב מ-KNN אך פחות מ-SVM.

מודלים עמוקים

כעת לאחר שהגענו לקצה של המודלים הקלאסיים, רצינו לנסות את המודלים המודרניים ולראות האם הם יצליחו להשיג ביצועים טובים יותר, למרות שכמות הדאטא לא גדולה במיוחד.

רצינו לבדוק מה יותר עוזר כדי לסווג את הספרות- **קשרים ארוכי טווח לעומת קשרים קרובי טווח**. לדוגמה- עבור קשר ארוך טווח, יש לזכור שנשמע \S\ בתחילת הספרה ולבסוף נשמע \N\ כדי לסווג SEVEN, לעומת זאת מקשר קצר טווח, אם שמענו \V\ ומיד אחריו \E\ (שיצר את הצליל VE) נוכל לסווג SEVEN.

המודל הראשון שעלה הוא RNN, בפרט LSTM. המודל LSTM מסוגל ללמוד גם קשרים ארוכי טווח וגם קשרים קצרי טווח. בנוסף נרצה מודל שמסוגל רק לקשרים קצרי טווח עבור זאת נשתמש בCNN, ואז נוכל להשוות ביניהם ולראות כמה כל קשר אינפורמטיבי (מועיל)...

נתחיל מ**LSTM** (Long Short-Term Memory) הוא סוג של RNN (Recurrent Neural Network) שמותאם במיוחד ללמידה של **תלות סדרתית** לאורך זמן.

איך זה רלוונטי לאודיו:

אות דיבור הוא **time series** – בכל רגע משתנה עוצמה בתדרים שונים. המידע החשוב לזיהוי ספרה מתפרס על פני שניות בודדות.

לדוגמה: המילה "seven" באנגלית כוללת רצף צלילים: תחילה עיצור /s/, אחריו תנועה /e/, ואז /v/ ועוד (נשים לב שבדאטא עצמו כל איבר הוא לא בדיוק עיצור אלא פונקציה בחלון, אך זהו אותו עיקרון). כדי לזהות אותה נכון, המודל צריך "לזכור" את הצליל מהתחלה גם אחרי שניות בודדות, כלומר ללמוד **תלות ארוכת טווח**. LSTM יודע לא רק להסתכל על פריים בודד, אלא לשמור בהדרגה הקשרים מהפריימים הקודמים. בכך אם הוא מזהה שנאמר S ואז E הוא יודע ללמוד שזה מעיד על SEVEN, גם אם זה מופיע בזמנים שונים. מה שרלוונטי הוא הסדר, כלומר שהופיע S ורק אז E ולא הזמנים הספיציפיים שבהם הם הופיעו (לדוגמה בשנייה 0.1 או בשנייה 0.4).

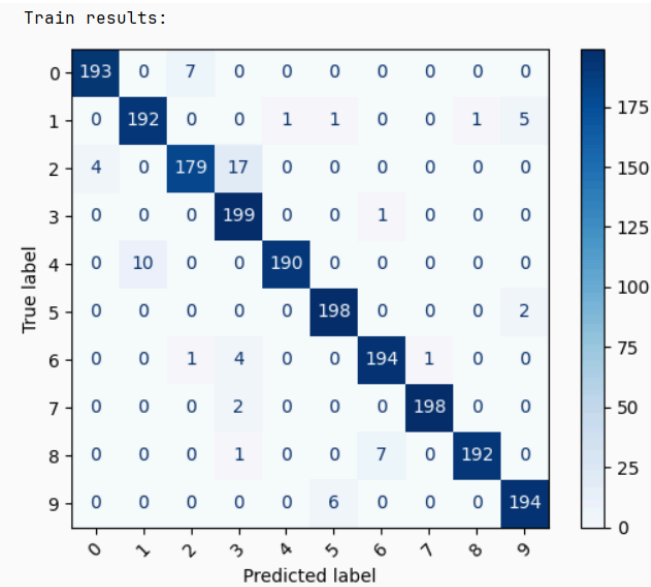
סיכום: LSTM מתאים כי הוא מאפשר למודל **לקשר בין חלקי ההקלטה** לאורך זמן, ולא רק להסתמך על תבנית מקומית.

החסרון העיקרי הוא שLSTM דורש הרבה מידע כדי להתאמן כראוי ולהיות מסוגל להכליל את עצמו מעבר לאימון.

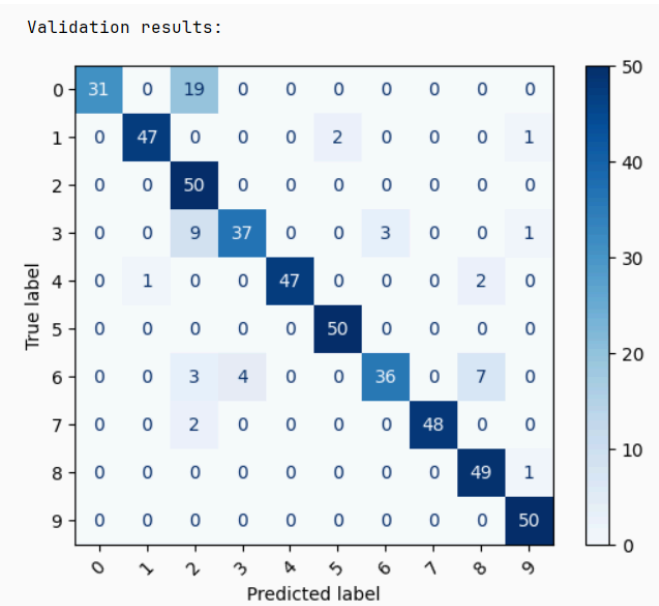
ט"ו

תוצאות LSTM:

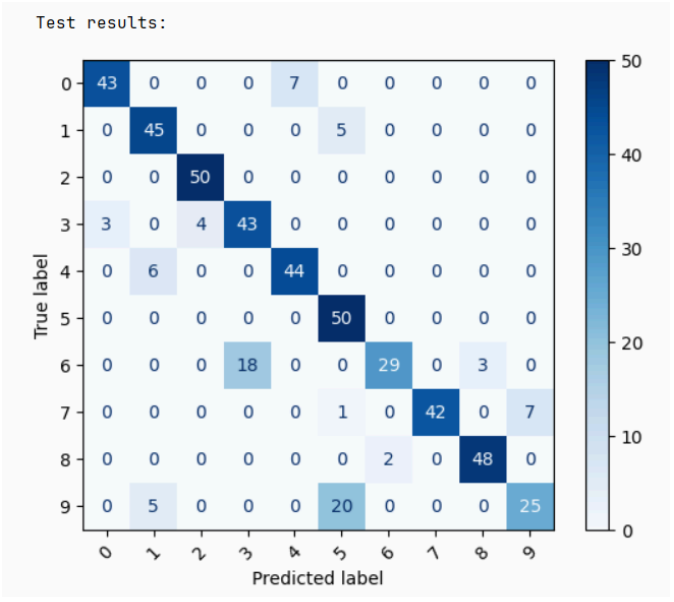
האימון:



ולידציה:



המבחן:



Metric	Train	val	Test
accuracy	0.9645	0.8900	0.8380
precision	0.9656	0.9157	0.8548
recall	0.9645	0.8900	0.8380
f1	0.9645	0.8906	0.8335

הLSTM דרש שינויים בארכיטקטורה (מספר שכבות hidden, גודל dropout), הוספת regularisation וכו' על מנת להלחם בoverfitting. לכן גם מדדי הtrain שלו נמוכים בקצת מהמודלים הקודמים. לאחר שינויים אלו, הצלחנו להגיע לתוצאות מוצלחות ביחס לשאר המודלים: 89% בvalidation ו83% בטסט. נסיף כי ההvalidation שומש עבור early stopping ולכן המודל לא ראה אותו לפני. על כן, זה עדיין אינדקציה לטסט ועל כן השגת תוצאות גבוהות משמעותית גם בvalidation וגם בטסט, מעידות על שיפור משמעותי מהמודלים הקודמים.

התוצאות הטובות יותר מעידות שאכן הקשרים הסדרתיים ארוכי טווח מועילים מאוד לסיווג הספרות מקטעי קול, ובכך אפילו מעט דאטה עם מימדים גבוהים הספיק עבור תוצאות טובות.

כעת נבדוק כמה קשרים קצרי-טווח אינפורמטיביים באמצעות CNN.

CNN (Convolutional Neural Network)–

מותאם לזיהוי דפוסים סביבתיים – תבניות שחוזרות על עצמן במטריצה (למשל תמונה).

איך זה רלוונטי לאודיו:

אם נסתכל על מקדמי MFCC כעל מטריצה (כל עמודה מסמלת חלון שונה וכל שורה היא מקדם שונה), אז ספרות שונות מייצרות תבניות שונות בתמונה הזו, ובאותו הזמן, דפוסים סביבתיים כמו לדוגמה עלייה בטון מעבר מצליל V לצליל E יראו דומה במטריצות של ההקלטות השונות.

לדוגמה, הספרה "seven" תראה במטריצה בתור המקדמים של הפונקציות בחלונות, והמודל יכול לראות את המעברים בין החלונות כמו שאנו שומעים את המעברים בקול.

חוזקות של CNN על אודיו

- CNN לומד פילטרים שמזהים תבניות כאלו – למשל "פס אנרגיה עולה", או "תדר נמוך קבוע לאורך זמן", שברמה גבוהה יותר, ניתן להסתכל על כך לדוגמה בתור מעבר בין עיצורים ותנועות.
- גם אם הדובר שינה קצת את הקצב או המבטא או הזמן שבו התחיל לדבר, ה-CNN יצליח לזהות את התבנית, בזכות **invariance מקומית** (יכולת לזהות משהו גם אם הוא הוזז מעט).
- באמצעות שכבות max pooling ניתן לזהות דפוס בסביבה כללית יותר, והמודל יידע להשתמש בזה שהדפוס הופיע באיזור הזה גם אם הוא לא הופיע במקום ספציפי. לדוגמה, אם נאמר VE בשנייה 0.1 או 0.2 אז השכבה לוקחת את המקסימום וזה יראה זהה.

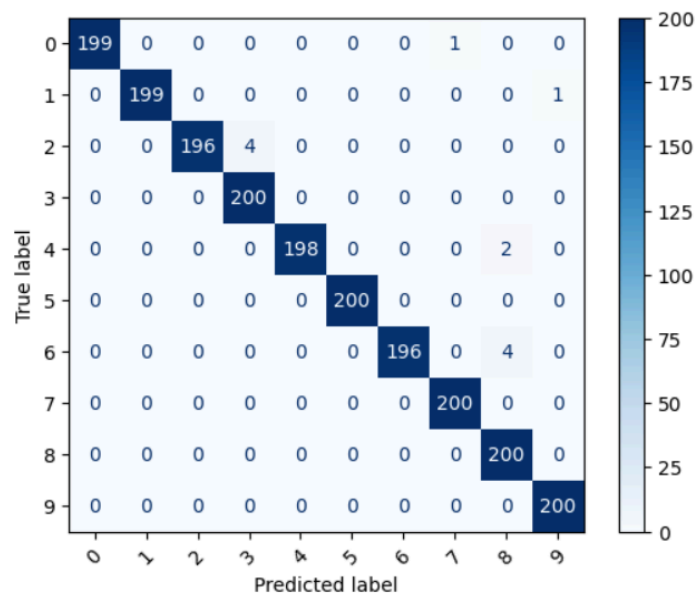
סיכום: CNN מתאים כי הוא מאפשר למודל לתפוס תבניות מקומיות, שמייצגות מאפיינים ייחודיים של ספרות.

בנוסף, CNN דורשת פחות דאטה על מנת להתכנס לתוצאה טובה ביחס ל-LSTM ולכן יותר סביר שה-CNN תדרוש פחות רגולריזציה.

ט"ו

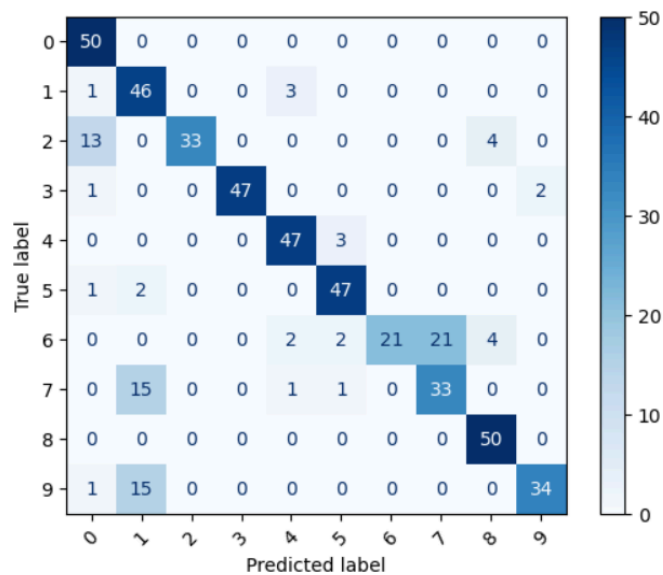
אימון:

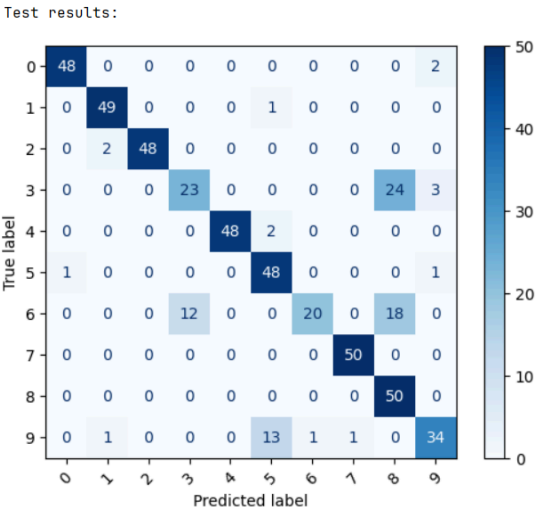
Train results:



ולידציה:

Validation results:





Metric	Train	val	Test
accuracy	0.9940	0.8160	0.8360
precision	0.9941	0.8527	0.8655
recall	0.9940	0.8160	0.8360
f1	0.9940	0.8106	0.8286

גם פה, ה-CNN השיג את תוצאות מוצלחות יותר ביחס למודלים הקלאסיים. ולכן ניתן להסיק כי גם קשרים קצרי-טווח אינפורמטיביים ועוזרים לסווג נכון את הספרות גם עם כמות דאטה לא גדולה.

כעת נשווה בין ה-CNN ל-LSTM:

מודל	Train Accuracy	Validation Accuracy	Test Accuracy
LSTM	0.9645	0.89	0.838
CNN	0.9940	0.816	0.836

מאחר ובאמת נדרש פחות רגולריזציה עבור ה-CNN, השגנו כעת תוצאות טובות יותר ב-train ב-CNN מאשר ה-LSTM. כמו קודם, ה-validation שומש עבור early stopping והמודל לא התאמן עליו.

המודלים השיגו תוצאות קרובות מאוד בטסט, וב-validation ה-LSTM סיווג טוב יותר. מכך ניתן להסיק, שייתכן שהקשרים ארוכי טווח עדיין מועילים, אבל לא מספיק עבור הפרש משמעותי. על פי התוצאות, קשרים קצרי-טווח אותם שני המודלים מנצלים, מביאים לתוצאות טובות והוספה של התייחסות לארוכי טווח לא מביאה לשיפור משמעותי.

סיכום

לסיכום, נראה את דיוק המודלים על הטסט של דוברים לא מוכרים, כדי לראות את ההתקדמות בין המודלים במשימה של סיווג ספרות מקטעי קול של דוברים חדשים:

LSTM	CNN	SVM	Random Forest	KNN	Test Accuracy
0.838	0.836	0.695	0.663	0.43	

כפי שניתן לראות בטבלה המסכמת, הביצועים של המודלים מסתדרים בסדר ברור, מהפשוט לחלש יותר ועד המורכב והמתאים ביותר למשימה:

- **(KNN 0.43)** – המודל הפשוט ביותר, שתלוי ישירות במדד מרחק בין דוגמאות. מאחר שהדאטא נמצא במימד גבוה מאוד וכולל שונות גדולה בין דוברים, מדדי המרחק הופכים פחות אינפורמטיביים (*curse of dimensionality*). לכן מצופה שKNN ישיג את התוצאה הנמוכה ביותר.
 - **(Random Forest 0.663)** – כבר מצליח לנצל אינטראקציות לא ליניאריות בין הפיצ'רים, אבל עדיין סובל מכך שחלק גדול מהפיצ'רים אינם אינפורמטיביים. השיפור המשמעותי לעומת KNN נובע מהיכולת "לבחור" תתי-קבוצות של פיצ'רים, אבל עדיין קשה להכליל על דוברים חדשים.
 - **(SVM 0.695)** – מותאם במיוחד למימד גבוה, יודע למצוא היפר-מרחב שמפריד בין הקלאסים גם אם רק חלק מהפיצ'רים אינפורמטיביים. זו כנראה הסיבה שהוא עקף את ה-Random Forest. עם זאת, בגלל שהדאטא רועש ויש שונות גדולה בין הדוברים, ההפרדה אינה מספיקה כדי להגיע לדיוקים גבוהים מאוד.
 - **(CNN 0.836)** – מצליח לנצל את מבנה המטריצה (MFCC לאורך זמן), ולזהות תבניות מקומיות שחוזרות על עצמן גם אם הן מוזזות מעט בזמן. היכולת הזו מאפשרת לו להכליל טוב יותר לדוברים חדשים. השיפור המשמעותי לעומת המודלים הקלאסיים נובע מהתאמתו הטבעית לנתוני MFCC-זמן.
 - **(LSTM 0.838)** – קרוב מאוד ל-CNN, אך עם יתרון קטן ביכולת להסתמך גם על קשרים סדרתיים ארוכי טווח. הדבר מועיל במקרים בהם זיהוי הספרה תלוי ברצף הצלילים כולו, ולא רק בתבניות מקומיות. למרות זאת, בגלל כמות הדאטא המוגבלת, היתרון של LSTM על CNN כמעט ולא בא לידי ביטוי, ולכן התוצאות קרובות מאוד.
- הסדר של המודלים משקף את יכולתם לנצל את המבנה האמיתי של הדאטא. ככל שהמודל יודע להסתמך פחות על מרחק גולמי במרחב מימדי, ויותר על תבניות מבניות (תדר-זמן) או קשרים סדרתיים (דיבור לאורך זמן), כך הביצועים משתפרים.

Aggregate לעומת Flatten:

כאשר השתמשנו ב-Aggregate (ממוצע וסטיית תקן), המודלים הצליחו טוב יחסית, אך איבדו מידע סדרתי. המעבר ל-Flatten, שבו כל פריים נשמר כרכיב בוקטור ארוך, שיפר את התוצאות משמעותית. הדבר מראה שהמידע שנמצא ברצף המלא של ההקלטה תורם הרבה יותר מאשר סיכום סטטיסטי בלבד, גם אם המחיר הוא מימדיות גבוהה מאוד. יחד עם זאת, ראינו שמודלים שונים מתמודדים עם המימדיות בצורה שונה: SVM ו-Random Forest הרוויחו הרבה יותר מ-KNN, שסבל מקללת המימדיות.

קשרים קצרי-טווח לעומת ארוכי-טווח:

בשלב הבא עברנו למודלים נירוניים (CNN ו-LSTM), שמנצלים את המבנה המטריציוני/סדרתי של חלונות ה-MFCC.

- ה-CNN התמקד בזיהוי תבניות מקומיות וקצרות טווח, והצליח להכליל היטב גם לדוברים חדשים, בזכות היכולת לזהות תבניות שחוזרות גם אם הן הוזזו מעט בזמן.
- ה-LSTM התמקד בלכידת קשרים סדרתיים גם ארוכי טווח לאורך כל ההקלטה, דבר שמתאים במיוחד לדיבור. למרות זאת, בגלל מגבלת כמות הנתונים, היתרון של LSTM לא בא לידי ביטוי במלואו, והתוצאות היו דומות מאוד ל-CNN.

מסקנות סופיות:

1. **ככל שהייצוג שומר על יותר מידע סדרתי ומבני, כך התוצאות משתפרות.** המעבר מ-Aggregate ל-Flatten שיפר ביצועים, והמעבר למודלים נירוניים שידעו לנצל את המבנה שיפר עוד יותר.
2. **מודלים שונים נותנים נקודת מבט שונה:** קלאסיים (KNN, SVM, RF) מדגימים את היתרונות והמגבלות של למידה על פיצ'רים סטטיסטיים (ממוצע + סטיית תקן) או על מרחבים גבוהי מימד, בעוד מודלים נירוניים (CNN, LSTM) מצליחים לנצל את הדינמיקה והמבנה של הדיבור.
3. **קשרים קצרי טווח מספיקים כדי להגיע לביצועים גבוהים יחסית,** וכנראה עיקר המידע להפרדה בין ספרות נמצא במאפיינים מקומיים, בעוד קשרים ארוכי טווח מוסיפים תרומה מסוימת אך קטנה.
4. **מבחינת סדר המודלים:** $LSTM \approx CNN < SVM \approx RF < KNN$ – וזה משקף היטב את המעבר מפתרונות פשוטים שתלויים במרחקים, למודלים מורכבים שמנצלים מבנה ותלות סדרתית.

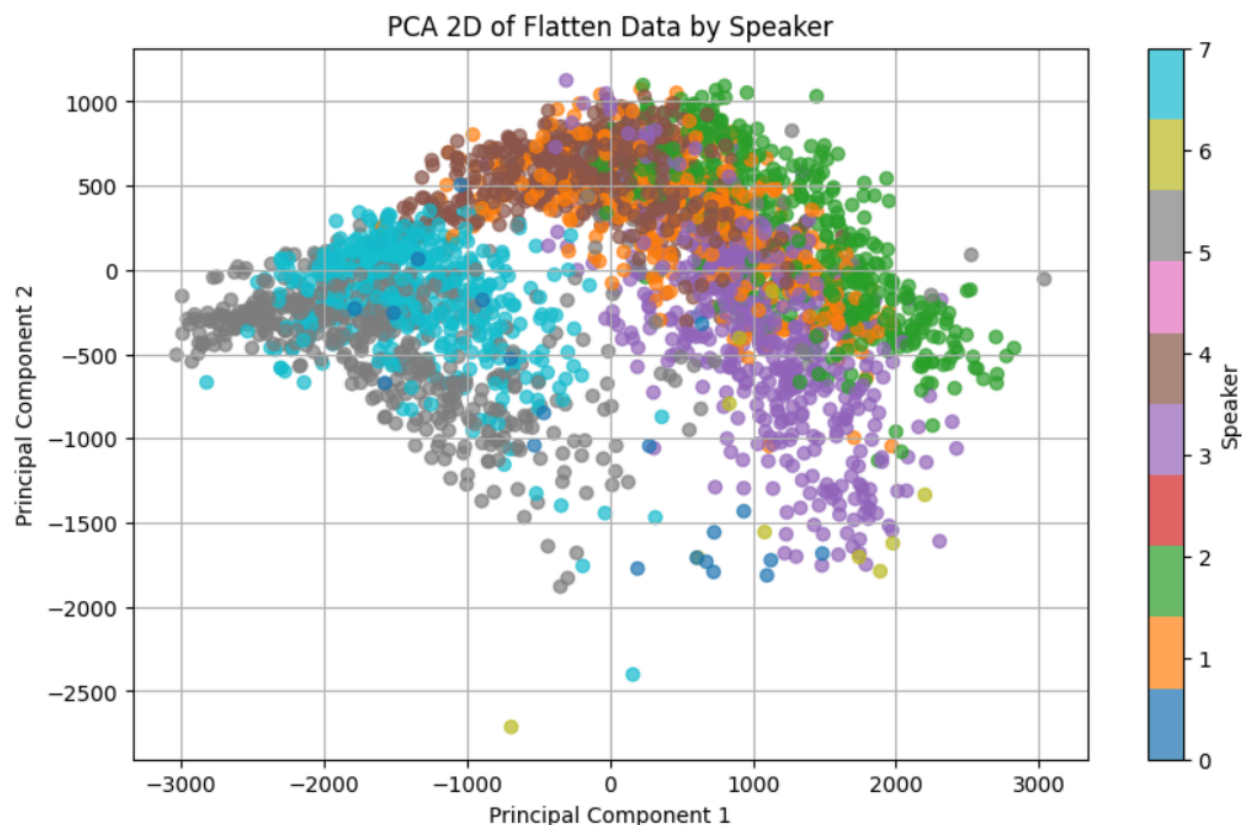
בונים 😊

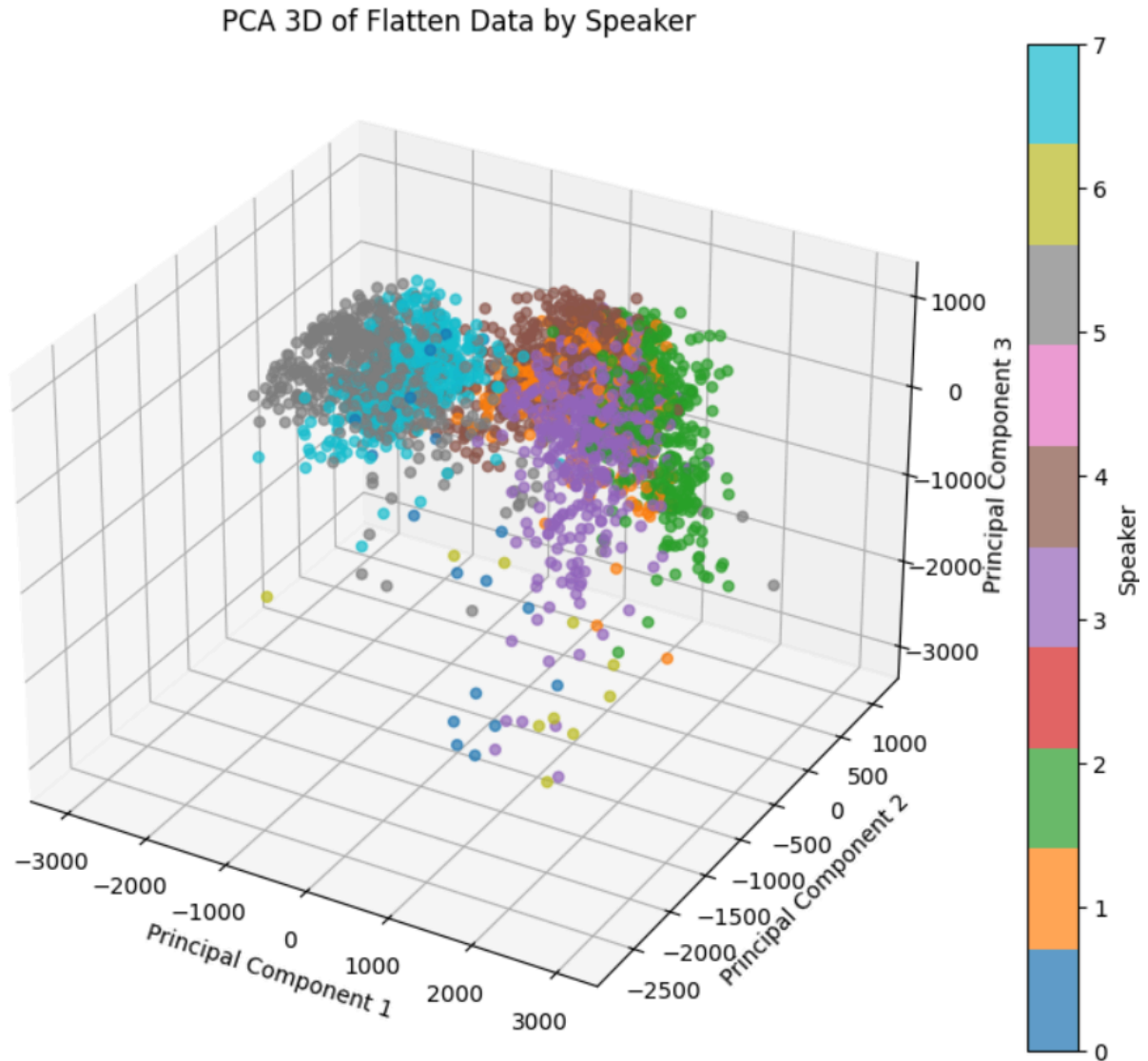
לאחר סוף העבודה, רצינו לבדוק לקחת את המודלים שבנינו בשביל ללמוד ולנסות אותם בעולם האמיתי, על הקולות שלנו. בכך, נוכל לראות כמה המודל באמת מצליח על אנשים בעולם האמיתי מעבר לdataset שהוא ריכוז של 6 אנשים בלבד (ומתוכם המודלים התאמנו על 4 בלבד). כלומר, גם אם ראינו שהמודלים מצליחים לסווג טוב דוברים לא מוכרים, מלכתחילה כל הדוברים היו גברים דומים יחסית עם תנאי הקלטה זהים. לכן זהו מבחן לכמה המודל מצליח מעבר לריכוז המקורי.

עבור כך, הקלטנו את יאיר אומר את 10 הספרות, ואת עדי אומר את 10 הספרות פעמיים, עם תנאי הקלטה שונים בין שני הסבבים. סה"כ 10 הקלטות של יאיר ו20 של עדי.

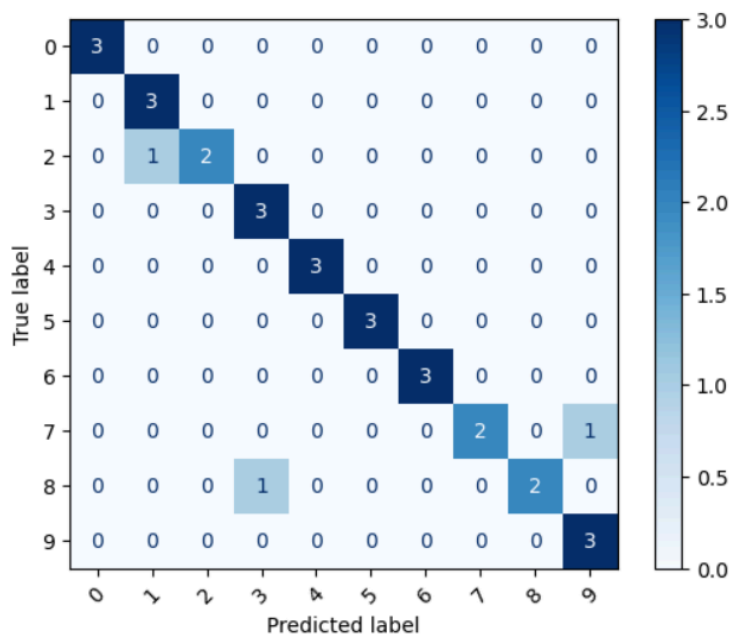
בניתוח הדאטא הקודם ראינו באמצעות PCA שבאמצעות MFCC, הדוברים מרוכזים, כלומר כל ההקלטות שלהם באותה הסביבה. נרצה לראות אם ההקלטות שלנו גם באותה הסביבה וכמה היא רחוקה או קרובה לסביבות האחרות. זה יעיד על כמה הדאטאסט כללי מלכתחילה.

בסרטוט הבא, המספר הסיידורי של עדי הוא 0 (כחול כהה) ושל יאיר 6 (צהוב).





ניתן לראות כי ההקלטות שלנו הרבה יותר מפוזרות, וגם רחוקות מהשאר. זה כנראה נובע גם מכך שהקלטנו פחות מ-500 הקלטות, אבל גם מעיד על כך שהתנאים של ההקלטות בדאטאסט כנראה לא טיפוסיות. לכן נצפה שהמודל פחות יצליח על הקלטות שלנו. נבדוק את המודל הטוב ביותר שאימנו - LSTM.



{'accuracy': 0.9, 'precision': 0.925, 'recall': 0.9, 'f1': 0.897142857142857}

למרות השוני שגרף PCA הראה, המודל השיג תוצאות גבוהות מאוד על הקולות שלנו! למעשה אפילו גבוהות יותר מבטסט. זה יכול להעיד על כך שההקלטות המקוריות היו יותר קשות לסיווג מאשר ההקלטות שלנו אבל כנראה גם נובע מכך שקבוצת הטסט שלנו קטנה בהרבה ולא כוללת יותר מדי מקרי קצה.